

Infrastructure as Code

Introduction

Ron C. Chiang
University of St. Thomas



About Your Instructor

- Associate Professor at University of St. Thomas
- Ph.D. George Washington University
- 20+ years in IT industry and academia
 - across multiple computing systems, such as cloud, mobile, embedded systems etc.
- AWS Certified
 - Cloud Practitioner
 - Solution Architect
 - AWS Academy Instructor
 - Educate Cloud Ambassador
- When I am not working, I enjoy the time with my family, travel, and outdoor activities.

Infrastructure

- What do we mean by “infrastructure”?
 - IT infrastructure provides all the necessary compute, storage, networking and software components necessary to deliver a service.

Traditional IT



Traditional IT

- Hand-built infrastructure based on physical hardware and guided by experience.
- Every infrastructure component is unique and special.
 - Built at different times with slightly different processes.
 - Built by different people with different levels of experience.
 - “Just make it work.”
- Infrastructure is rarely replaced and is fanatically supported throughout its lifecycle.
- Infrastructure changes are carefully controlled by a Change Advisory Board (CAB).

Traditional IT

- Role of IT operations staff:
 - Gatekeepers for all IT services in the organization.
 - “The Office of No.”
 - Often partitioned into specialist teams:
 - Network engineers
 - Storage engineers
 - Security analysts
 - DBAs
 - Ops team focused on preventing infrastructure failure.
 - “Keep the lights on.”

Traditional IT server builds

- The server build process:
 - Purchase request
 - Purchase order
 - Receive components
 - Build server
 - Test build/ update firmware
 - Install OS/Hypervisor
 - Test server
 - Rack server
 - Network provisioning
 - Storage provisioning
 - Firewall provisioning
 - Application server installation
 - Backup/ Monitoring provisioning
 - Disaster recovery provisioning
 - Security testing
 - Data migration
 - Application service testing

*1 to 6 months
later you have
a server!*



Modern IT



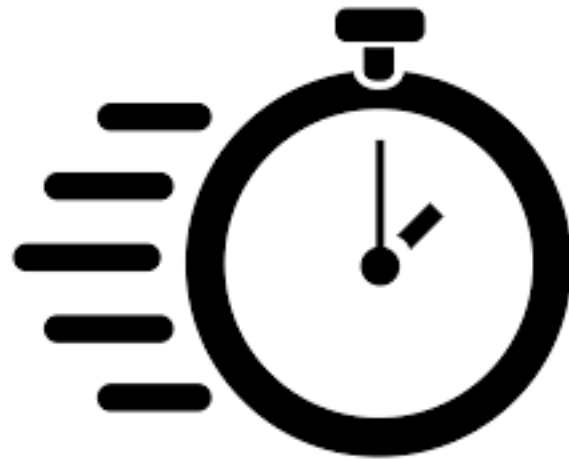
Modern IT

- Design automated infrastructure that is repeatable and testable.
 - Humans build the machines that build the infrastructure.
- Elimination of **snowflakes** – unique infrastructure components.
- Infrastructure is constantly being replaced.
 - “We embrace change.”
- Infrastructure changes are automated by a change management pipeline instead of a review board.

Modern cloud servers

- The cloud server build process:
 - Review current computing budget quotas
 - Select pre-validated infrastructure stack
 - Deploy stack into virtual data center
 - Automatically load application services and data

*2 minutes
later you
have a
server!*



What is Infrastructure as Code?

- an approach to infrastructure automation based on practices from software development.
- It emphasizes consistent, repeatable routines for provisioning and changing systems and their configuration.

What is Infrastructure as Code?(cont.)

- Infrastructure as Code (IaC) tools allow you to manage infrastructure with *configuration files* rather than through a graphical user interface.
- IaC allows you to build, change, and manage your infrastructure in a safe, consistent, and repeatable way by defining resource configurations that you can version, reuse, and share.

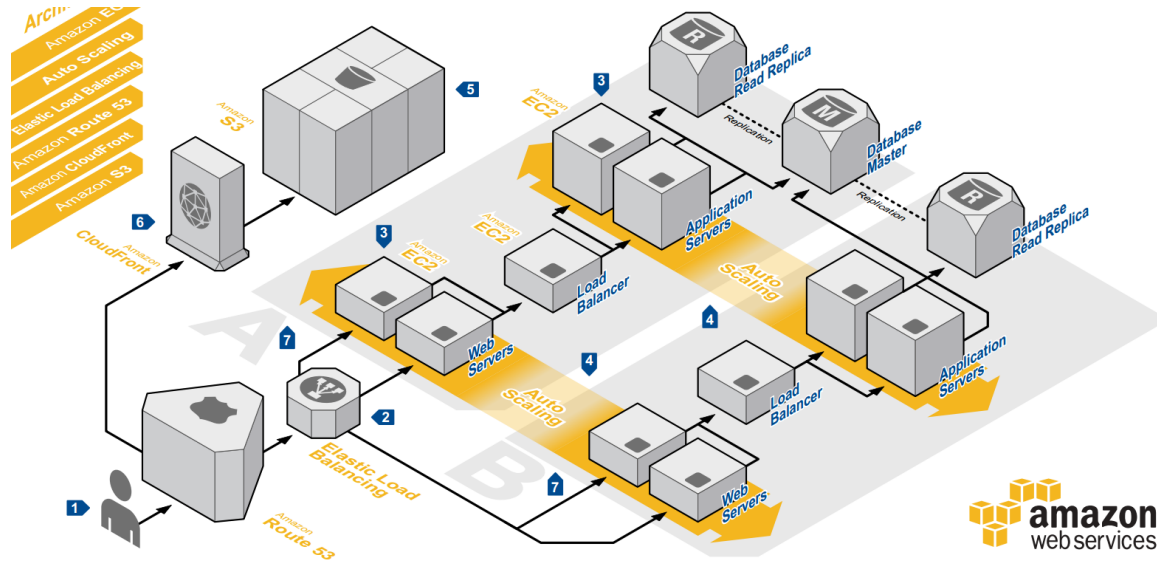
Benefits of Infrastructure as Code

- Using IT infrastructure as an enabler for rapid delivery of value
- Reducing the effort and risk of making changes to infrastructure
- Enabling users of infrastructure to get the resources they need, when they need it
- Providing common tooling across development, operations, and other stakeholders
- Creating systems that are reliable, secure, and cost-effective
- Make governance, security, and compliance controls visible
- Improving the speed to troubleshoot and resolve failures

Core Practices

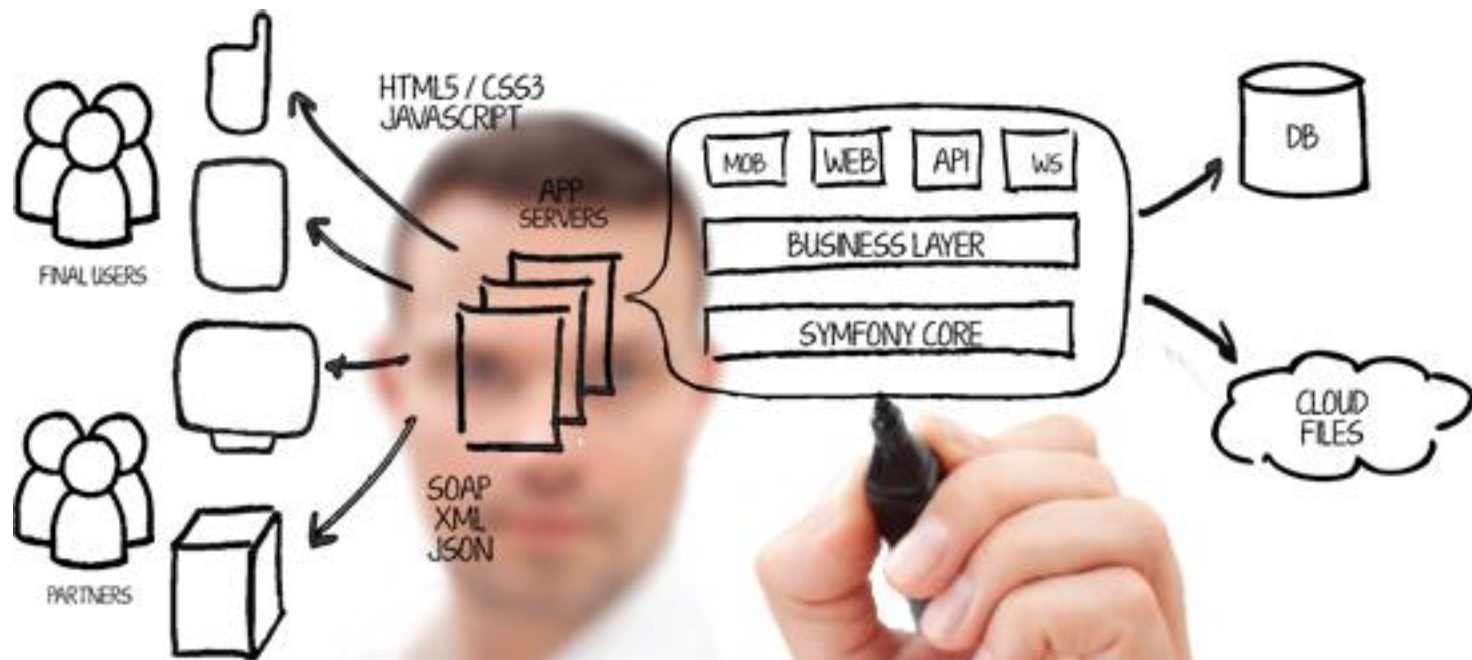
- Define everything as code
- Continuously test and deliver all work in progress
- Build small, simple pieces that you can change independently

Cloud Infrastructure



- Scripted, automated infrastructure available on demand from a global cloud computing service.

Course Syllabus



Source Control Management

Source Control Management

- Definition: “Version control, also known as source control, is the **management of changes to documents**, computer programs, large web sites, and other collections of information.” – *Wikipedia*
- In our class, source control is about maintaining a history of all the changes made to code files.

Source Control Management

- We use source control systems to:
 - Store and track changes in software source code and application configurations (or most any sort of text-based artifact)
 - Maintain different releases of software code
 - Effectively collaborate as a team on a single set of artifacts
- First systems was SCCS, written in 1972 by Rochkind at Bell Labs.
- Then came RCS, CVS, and Subversion
 - Dozens of commercial solutions available: Perforce, ClearCase, Microsoft TFS

Types of source control

- **Local**
 - Simple file versioning
 - RCS

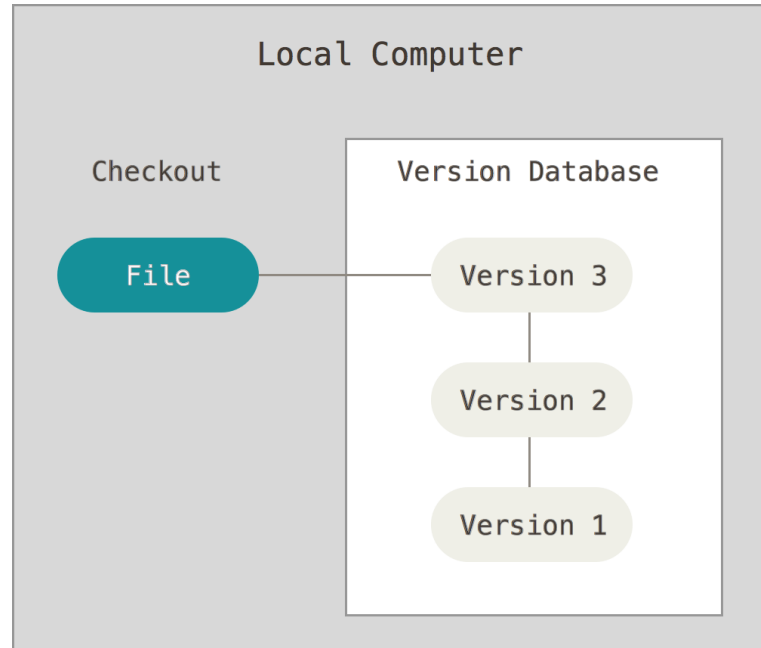


Image from <https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control>

Types of source control

- **Centralized**
 - Team approach
 - Everyone works on same files
 - Central server = single point of failure
 - Hard to branch
 - Example: Subversion

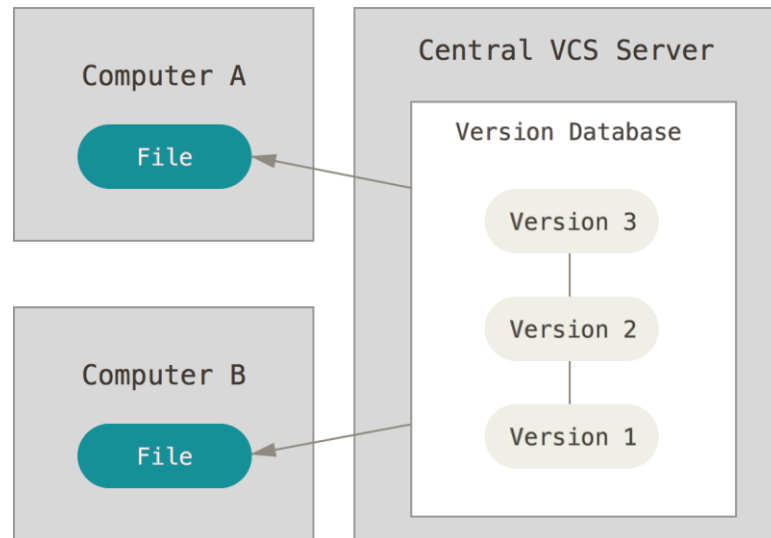


Image from <https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control>

Types of source control

- **Distributed**
 - Uses distributed team approach
 - Everyone works on their own set of files
 - Easy to branch
 - Need to handle merge conflicts
 - Example: Git

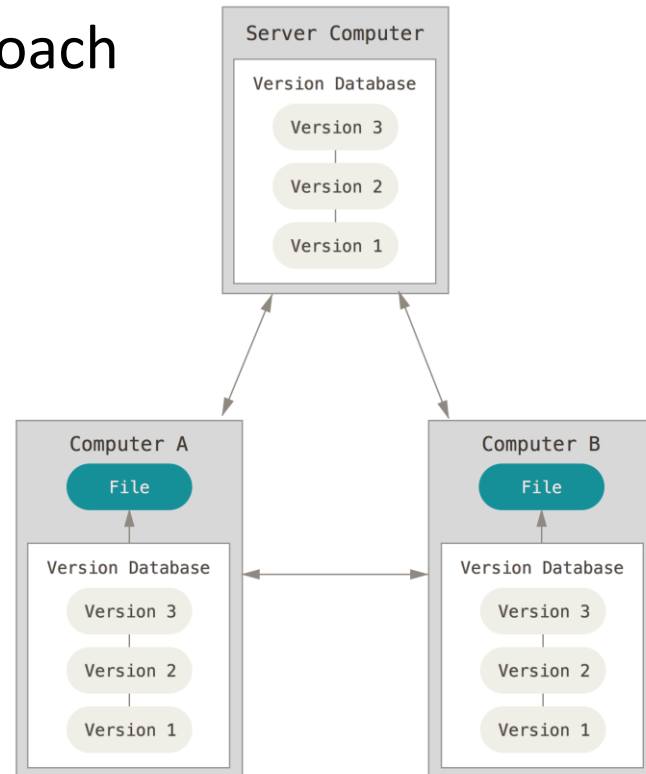


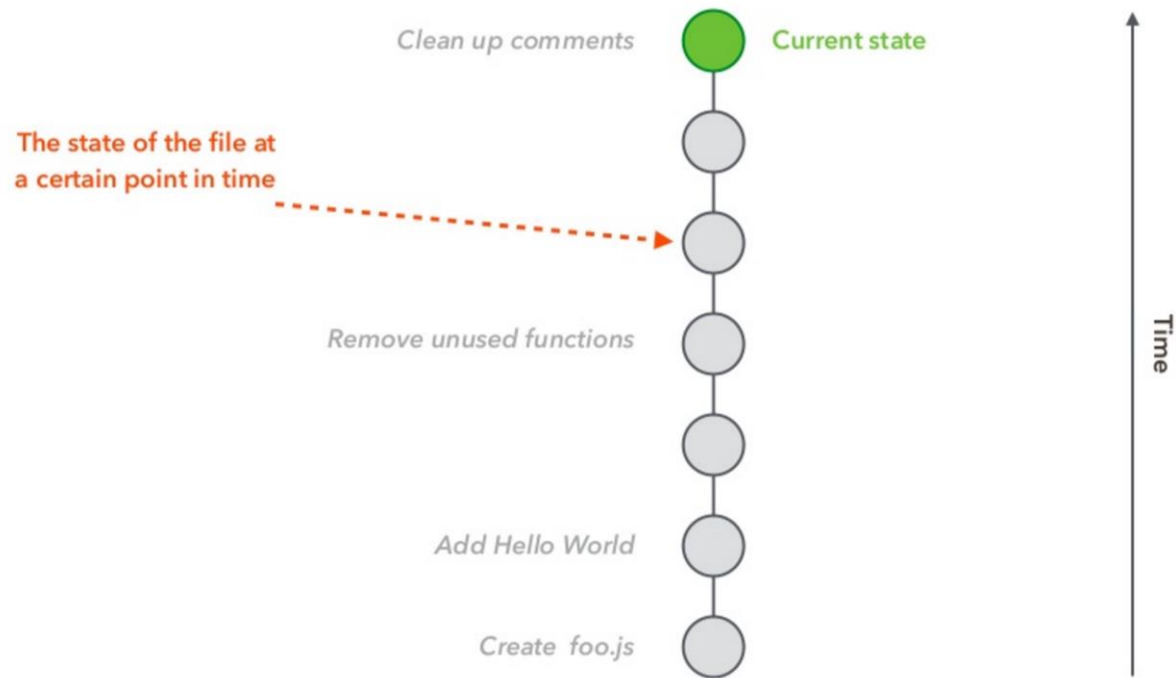
Image from <https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control>

Git

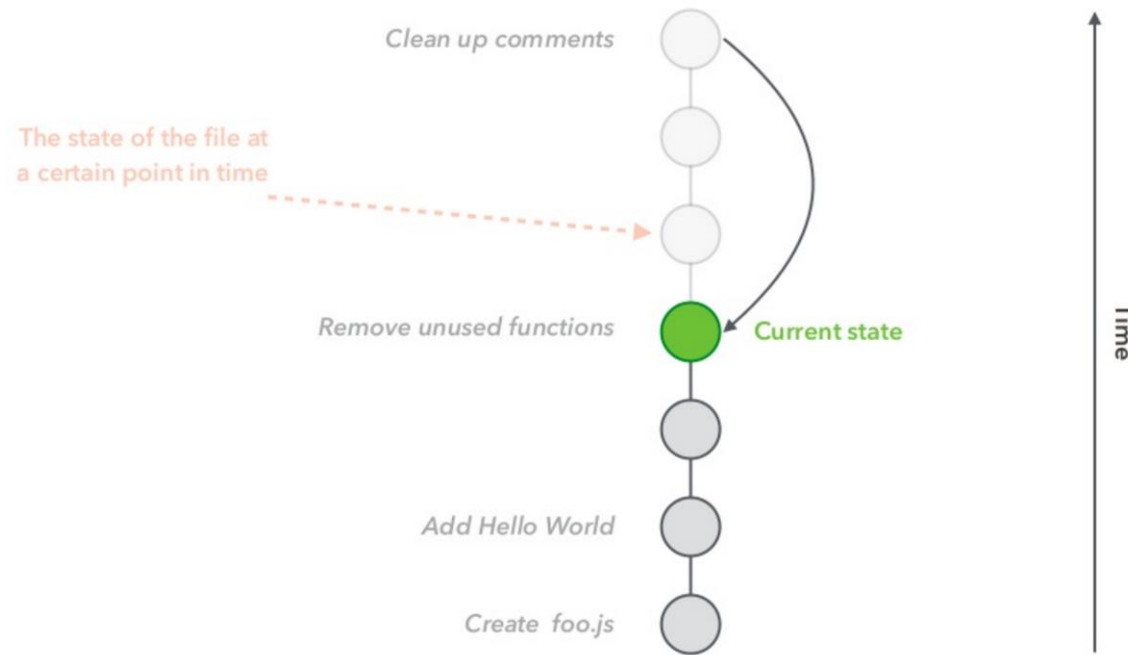


- Most popular open source distributed source control management system.
- Initially designed by Linus Torvalds (Linux) in 2005.
- Goals:
 - Super fast patching.
 - Supports one developer or thousands of developers.
 - Designed from a file-system perspective, not a source control management perspective.

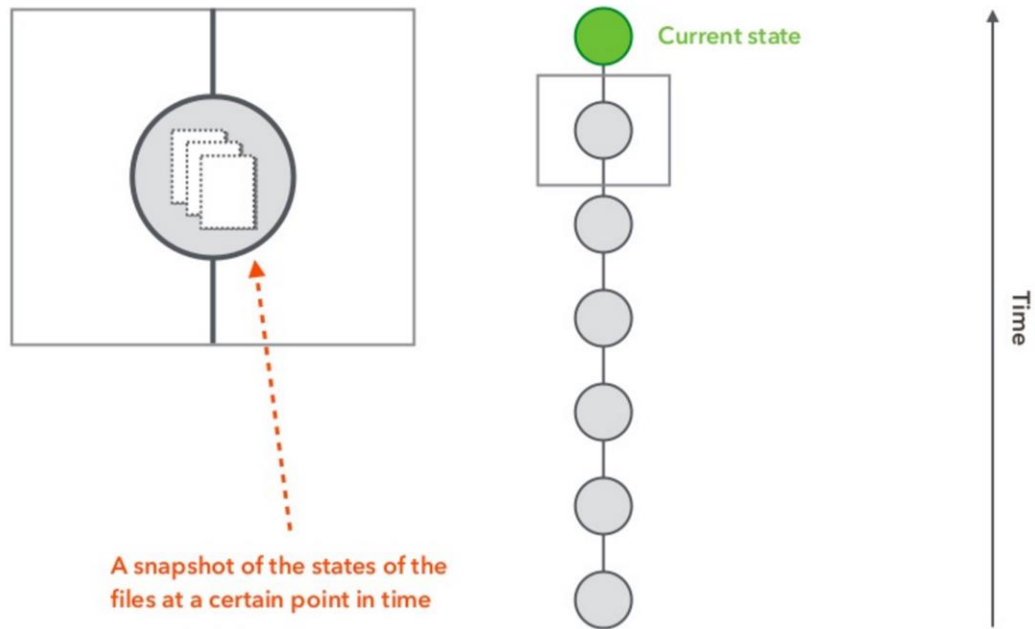
File



File



File(s)



Creating a Project

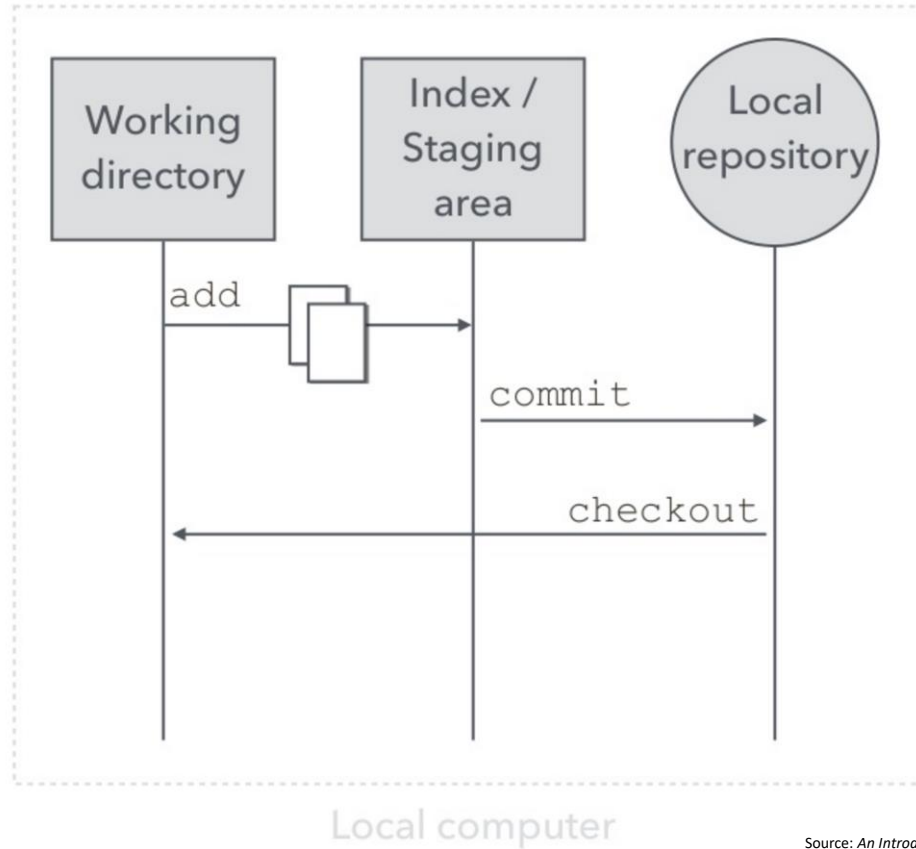
- Start a new project:

```
> git init
```

- Download an existing project:

```
> git clone <url>
```

How Git works

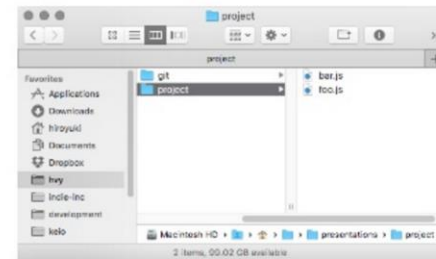


Source: *An Introduction to Git* by Hiroyuki Vincent Yamazaki

Add command

Tell Git that a file is to be
included in the next snapshot
by adding it to the index /
staging area

```
> git add foo.js
```

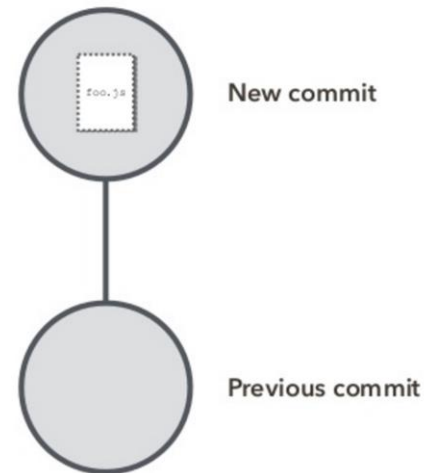


Commit command

Take a new snapshot of the state of the files in the staging area (append it to the previous commit)

Store a snapshot in the local repository

```
> git commit
```



How Git Works

- Git only creates new files when a file is modified.
- Git stores references to files.

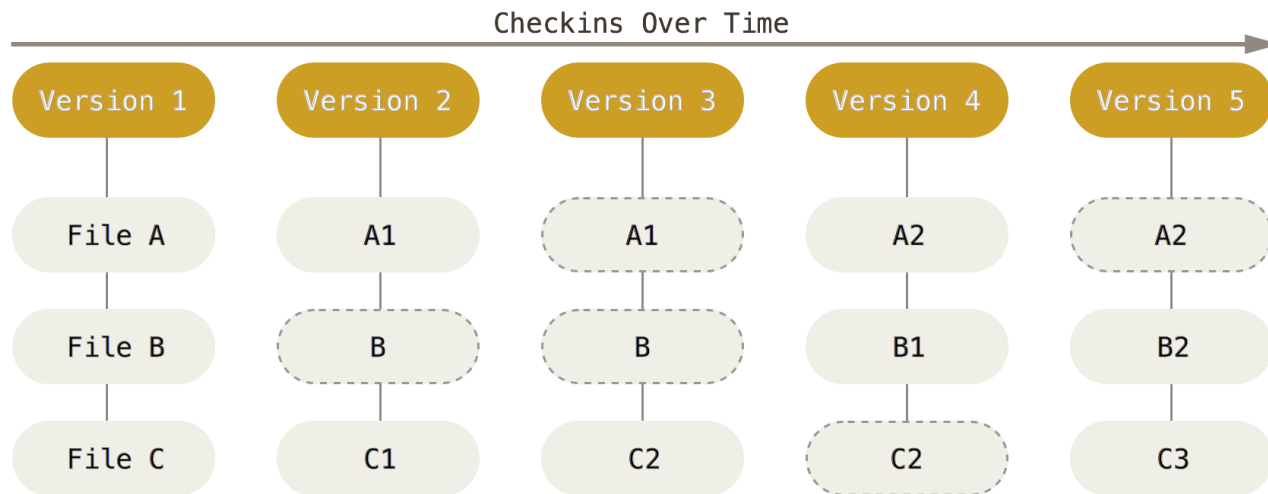


Image from <https://git-scm.com/book/en/v2/Getting-Started-Git-Basics>

Git Commits

- Commits contain:
 - SHA-1 checksum (Commit Hash)
 - Author
 - Date
 - Message
 - Reference to parent commit(s)
 - Tree
 - Contains references to all the files in the snapshot

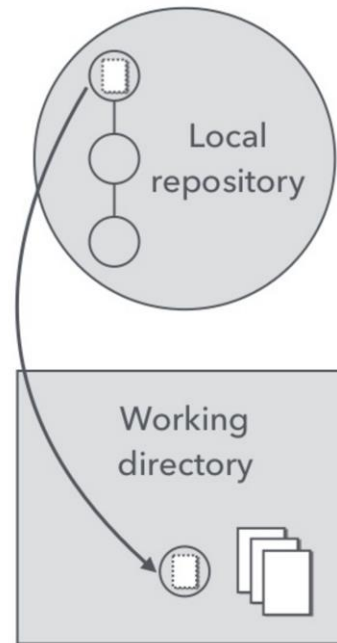
Source: *An Introduction to Git* by HiroYuki Vincent Yamazaki

Git Checkout

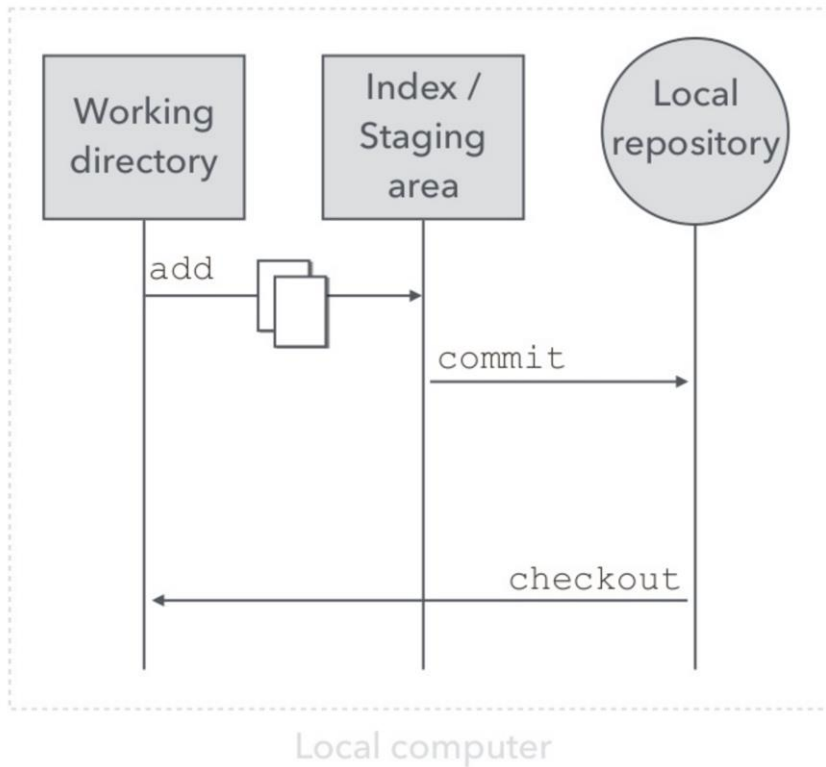
Get the data **from the repository into the working directory**

Again, not the files but the references to the files from a commit

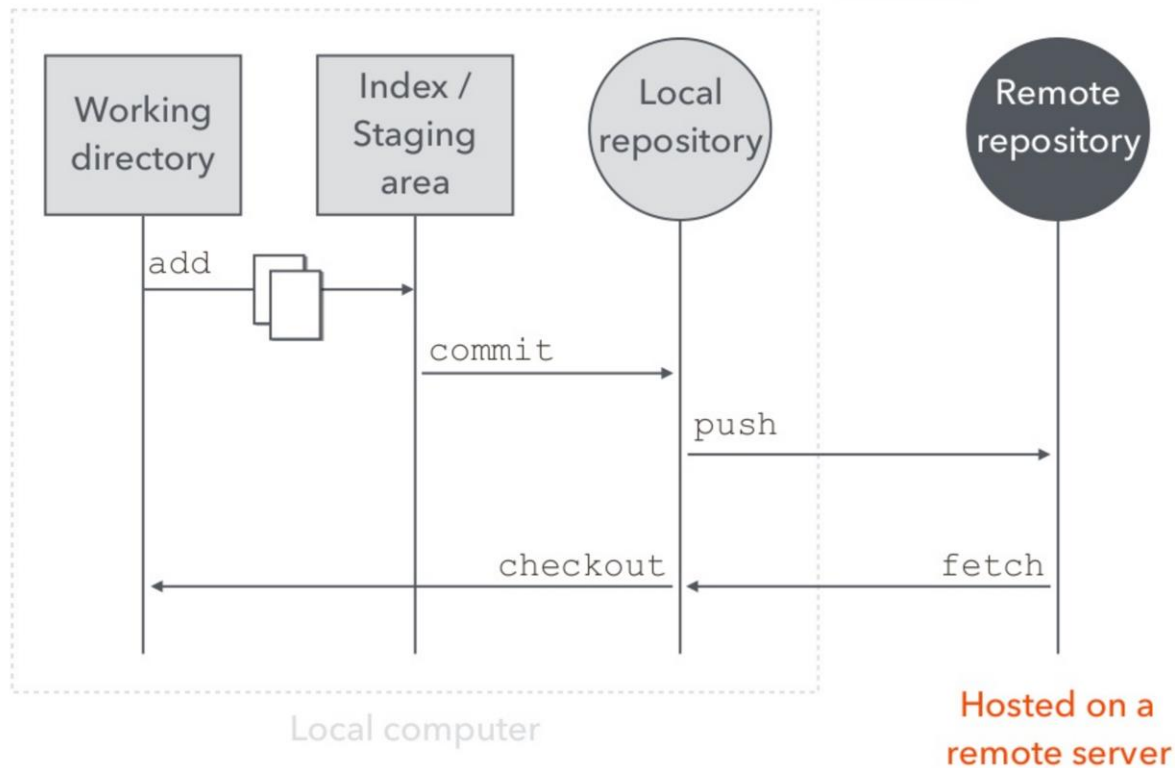
```
> git checkout <commit>
```



Git Collaboration



Git Collaboration



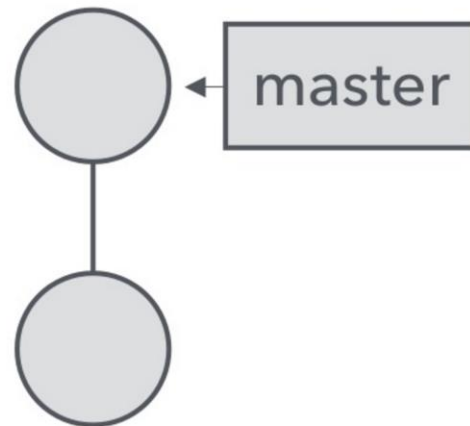
Remote Repository

- The remote repository is a copy of the local repository hosted on a network.
- Other people can **clone** the remote repository to download their own copy.
- GitHub provides a place to host a remote repository.

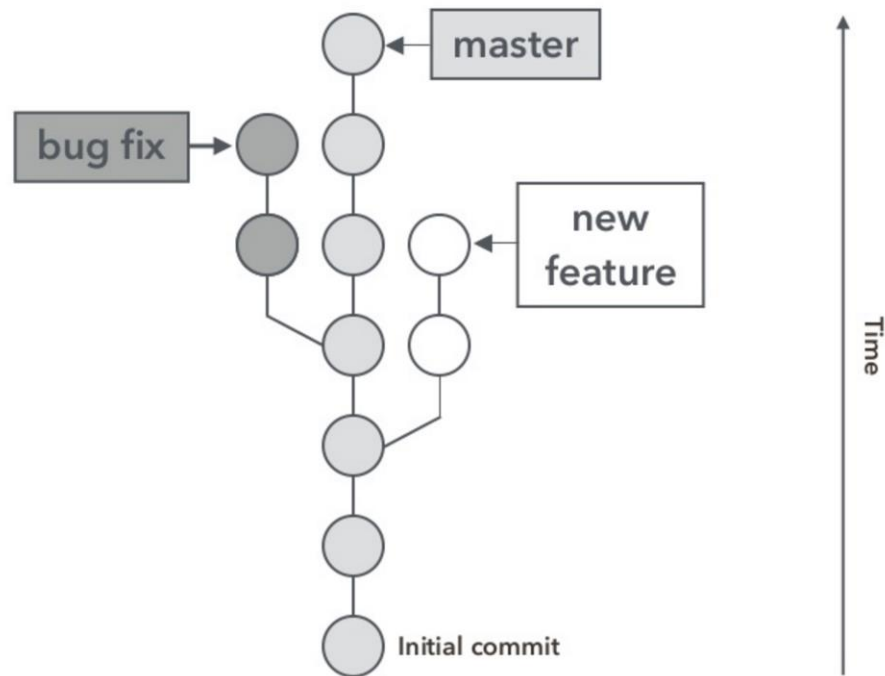
Git Branch

A branch is a **pointer** to a commit

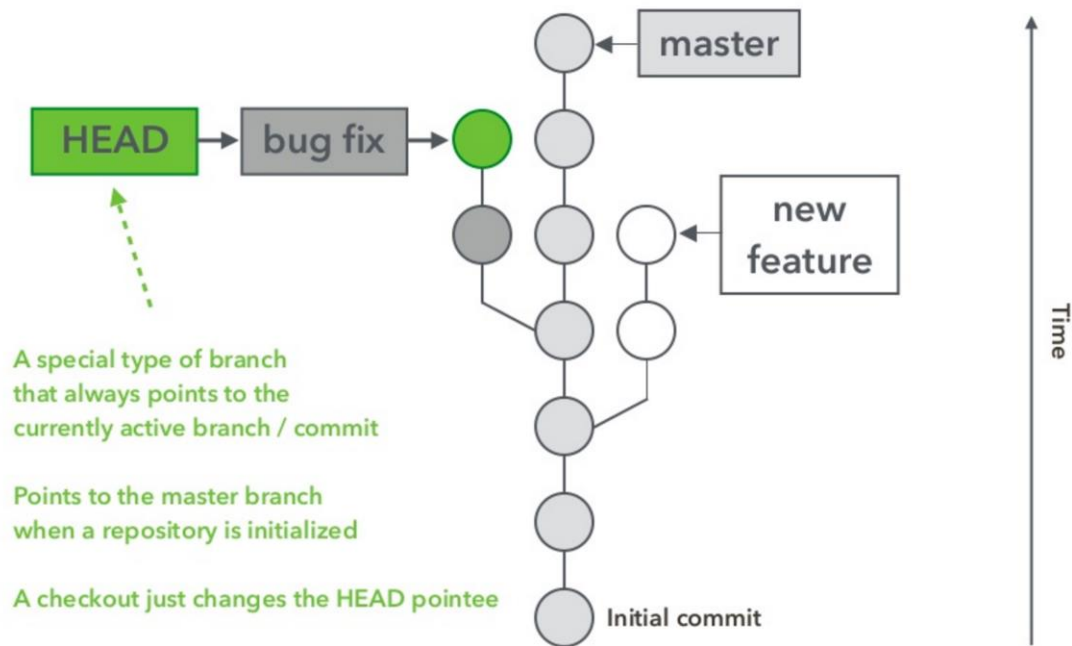
master, is the name of the default branch



Git Branch



Git Branch



Git Merge

Merge two commits together

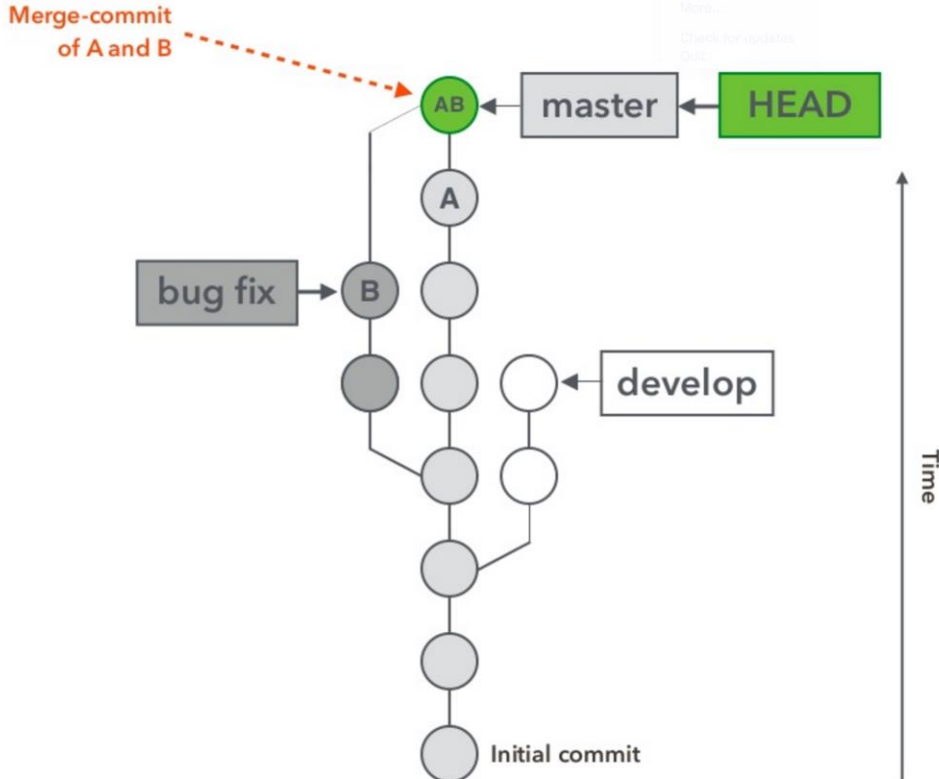
Can be used to merge branches

May cause **conflicts** between commits that need to be resolved manually

```
// Merge <commit> into HEAD pointee
```

```
> git merge <commit>
```

Git Merge



Source: *An Introduction to Git* by Hiroyuki Vincent Yamazaki

Git Hands-on

- The best way to learn Git is by using the git command using a command line interface (terminal shell)
- You can also use graphical Git clients such as Source Tree. Find more at <https://git-scm.com/downloads/guis>
 - These are very helpful when working with large teams
- Refer to Git Hands-On Guide
- Book: <https://git-scm.com/book/en/v2>

Git Hands-on

MacOS Commands:

```
mkdir project1
$ cd project1
$ git init
$ touch data.json
$ git status
$ git add data.json
$ git commit -m "my first commit"
```

```
$ touch nginx.conf
$ git status
$ git add nginx.conf
$ git commit -m "add web configuration file"
```

```
$ git checkout <1st commit hash>
$ git checkout <2nd commit hash>
```

```
$ git branch develop
$ git checkout develop
$ touch index.html
$ git add *
$ git commit -m "add homepage"
```

```
$ git checkout master
$ git merge develop master
```

Windows Commands:

```
$ mkdir project1
$ cd project1
$ git init
$ ECHO >> data.json
$ git status
$ git add data.json
$ git commit -m "my first commit"
```

```
$ ECHO >> nginx.conf
$ git status
$ git add nginx.conf
$ git commit -m "add web configuration file"
```

```
$ git checkout <1st commit hash>
$ git checkout <2nd commit hash>
```

```
$ git branch develop
$ git checkout develop
$ ECHO >> index.html
$ git add *
$ git commit -m "add homepage"
```

```
$ git checkout master
$ git merge develop master
```

Infrastructure as Code

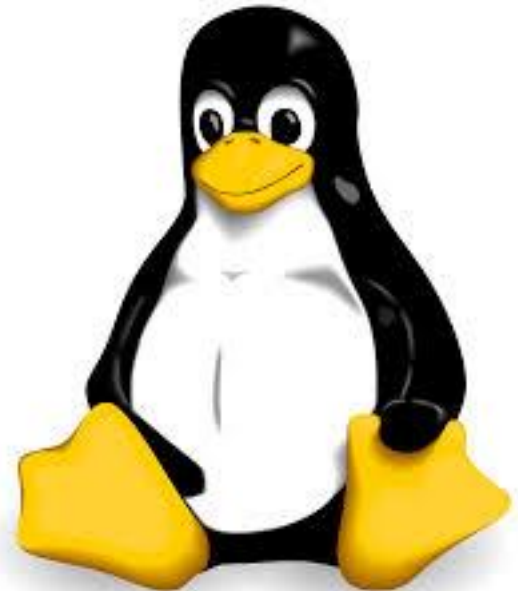
Linux CLI and Shell Scripts

University of St. Thomas
St. Paul, MN



Agenda

- Linux fundamentals
- Package management
- Automation
- Shell scripting



Linux



- History
 - Started by Linus Torvalds in 1991
 - Inspired by Unix, developed at AT&T in 1970s
 - Linux is not Unix, but uses same concepts.
 - Open Source Software (General Public License)
- Runs on cells phones and the largest supercomputers.
- Why Linux?
 - Common platform for modern distributed applications.
 - Automation starts at the command line, not the GUI.

Linux

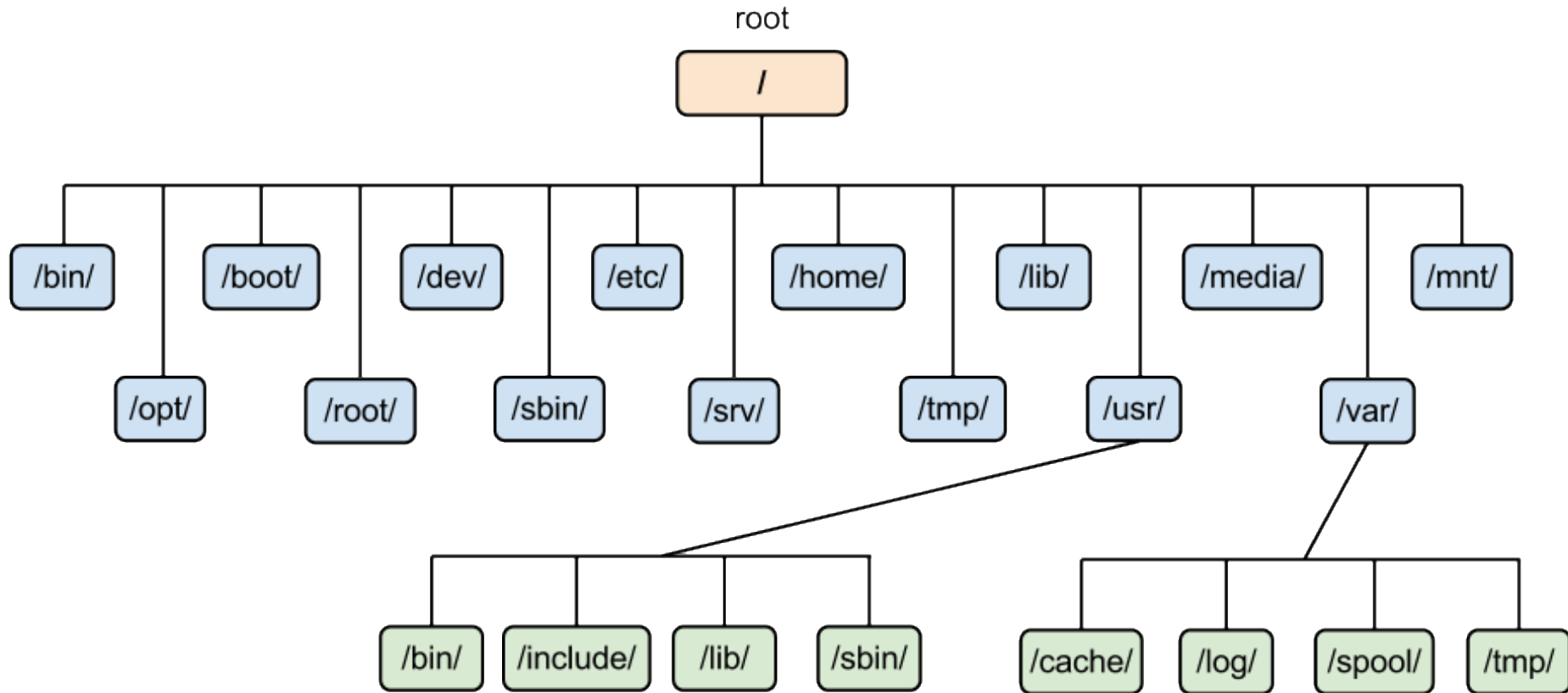
- A Linux **distribution** includes a kernel and a collection of applications.
 - Linux kernel
 - Applications (GNU, etc)
 - Desktop (Gnome, KDE, etc.)
- Dozens of distributions available to suit a large variety of needs.
 - RedHat, SUSE, Ubuntu, CentOS, Debian

Accessing Linux on AWS



- Requirements:
 - Amazon Linux EC2 instance running in a public subnet.
 - Public/Private access keys for the instance.
 - SSH Client:
 - Windows -> Git for Windows (gitforwindows.org)
 - MacOS -> Terminal (built-in) or iTerm2 (www.iterm2.com)
 - Linux vm -> xterm or other terminal

Linux File System Hierarchy



Linux File Types

Type		Description
Normal	-	Regular file
Directory	d	Regular directory
Symbolic link	l	Shortcut alias to a file or directory
Socket	s	Inter-process communications
Named pipe	p	Similar to socket, user cannot access
Character device	c	Hardware communications
Block device	b	Hardware communications

\$ ls -l

-rw-r--r--	ordinary file
drwxr-xr-x	directory file
brw-rw----	block device file
lrwxrwxrwx	symbolic link

Linux File Permissions

\$ ls -l

drwxr-xr-x 4 nits nits 4096 2011-12-26 01:55 Desktop

drwxr-xr-x

r - read
w - write
x - execute

Type **U**ser **G**roup **O**thers

Linux File Names

- File names on Linux are **case sensitive**.
 - So are commands because these are just executable files!
- Linux file names don't have dot extensions like Windows.
- A file name starting with a period (.) is called a hidden file and isn't displayed in a standard directory listing.

Linux Shell Shortcuts

- The BASH shell provides a simple, yet powerful command entry interface. Useful shortcuts include:
 - **Ctrl+a**: move the cursor to the beginning of the command line
 - **Ctrl+c**: terminate a running program and return to the shell prompt
 - **Ctrl+d**: log out of the current shell
 - **Ctrl+e**: move cursor to the end of the command line
 - **Ctrl+l**: clear the shell terminal screen
 - **Ctrl+r**: search the command history
 - **<tab>**: autocomplete file name
 - **<tab><tab>**: show command completion possibilities
 - **<up arrow>**: repeat last command (or scroll through history)

Linux Command Help

- Linux provides a couple different methods to get helpful information about a command:

```
$ man <command>
```

```
$ <command> --help
```

- Sometimes the best way to get help is just using a web search engine.

Linux User Accounts

- Each user on Linux has a separate account with a password.
 - We'll use the **ec2-user** account during this class.
- User accounts can be members of user groups.
- The root account is known as a super-user and is all powerful (like Administrator on Windows).
- Typically we log into a Linux system and escalate our privileges to become a super-user using the **sudo** command (super-user do).
 - Like "Run as Administrator" on Windows.

Linux User Accounts

- Users have a defined home directory, typically something like **/home/username**
 - The tilda (~) character is used as an alias for a user's home directory.
- You can add new users using the `useradd` command:
 - `$ useradd steve`
- And delete users using the `userdel` command:
 - `$ userdel -r michael`

Linux Directory

- The Linux file system is partitioned into separate directories, denoted by forward-slash character (/).
 - Tree structure starting with base (/) root directory.
- Current directory you are in is called the current working directory.
- Use the **pwd** command to print the current working directory and the **ls** command to list the contents of the directory.

Changing Directory

- The **cd** command is used to change the current working directory to a new one.
- Two options:
 - Specify an absolute path name:
 - `$ cd /home/steve/documents`
 - Specify a relative path name:
 - `$ cd documents`
 - change to documents sub-directory located in present directory
 - `$ cd ../databases` (change to databases
 - change to databases sub-directory located in parent directory
 - `$ cd ~`
 - change to user's home directory

Managing Directories

- Use the **mkdir** command to create a new directory.
 - \$ mkdir test
 - Creates a sub-directory called test in the present directory.
 - \$ mkdir /home/steve/test
 - Creates a sub-directory called test in the /home/steve directory.
- Use the **rmdir** command to remove a directory.
 - \$ rmdir test
- The rmdir command will fail if the directory isn't empty.
 - Another option: \$ rm -fr test (careful!)

Creating Files

- Easy way to create an empty text file is to use the **touch** command.
 - \$ **touch myfile.txt**
- You can also use one of the basic text editors to create a file.
 - \$ **nano myfile.txt** (recommended for beginners)
 - \$ **vi myfile.txt**

Viewing Files

- Linux provides a couple different tools to view the contents of text files.
 - **\$ cat myfile.txt**
 - **\$ more myfile.txt** (similar to cat but with pagination)
 - **\$ less myfile.txt** (less is more than more 😊)
- You can always open the file in a text editor (nano or vi).

Listing files

- The `ls` command allows you to list the files in a directory.
 - `$ ls`
 - `$ ls /home/ec2-user`
 - `$ ls ~`
- Add the `-la` option to the command to see more file details.
 - `$ ls -la`
 - `$ ls -la /home/ec2-user`

Managing Files

- Copy the file to another file location using the **copy** command.
 - \$ **cp myfile.txt myfile.bak**
- Move the file (rename) using the **mv** command.
 - \$ **mv myfile.txt myfile2.txt**
- Delete a file using the **rm** command.
 - \$ **rm myfile2.txt**

File Permissions

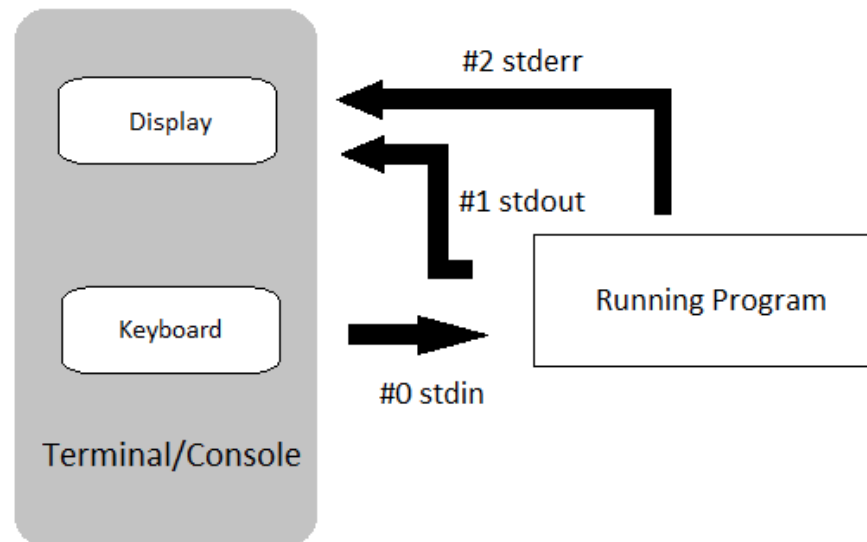
- Change the owner of a file using the **chown** command.
 - **\$ chown jbaker file1**
- Change the group associated with a file using the **chgrp** command.
 - **\$ chgrp webusers file1**
- Change the file permissions using the **chmod** command.
 - **\$ chmod a+r file1**
 - Set the permissions so that all users can read the file
 - **\$ chmod a+x file1**
 - Set the permissions so that everyone can execute the file (shell script).
- We can also use a numeric argument (octal) to set the file permissions:
 - **\$ chmod 600 file1**
 - Set read and write permissions for the file owner
 - **\$ chmod 775 file1**
 - Give everyone read+execute, owner and group get all access

File Globbing

- **Globbing** is the use of pathname expansion to refer to one or more files.
- Uses special characters to expand pathname:
 - * matches all characters (wildcard)
 - ? matches a single character
 - \$ **ls *.jpg** (list all jpeg files)
 - \$ **ls ?.jpg** (list jpeg files with 1 character names)
 - \$ **rm [A-Z]*.jpg** (remove jpeg files that start with capital letter)

I/O Redirection

- Linux programs accept input data from a keyboard and output to a terminal by default.
 - Data input path is called **stdin** (standard input).
 - Data output path is called **stdout** (standard output).



I/O Redirection

- It's possible to redirect the I/O pathways of linux programs.
- Redirect the output of a program using the right angle-bracket character (>).
 - **\$ ls /home/ec2-user > user-files.txt**
 - Write the output of the directory listing to the user-files.txt file.
- Append data to an existing file rather than overwriting it using two brackets (>>).
 - **\$ ls /var/www/html >> htmlfiles.txt**
- Redirect the input of a program using the left angle-bracket character (<).
 - **\$ wc -l < file.txt**
 - Input the file.txt file into the wordcount program to count the number of lines.

Command Pipelines

- A command pipe allows you to take the stdout (output) of a command and send it to the stdin (input) of another command.
- A pipe is denoted using the vertical bar character (|).
 - **\$ ls /var/www/html | sort**
 - List the files /var/www/html directory and output the listing in alphabetical order.

Processes

- Each program on Linux runs as a processes or set of processes.
 - Some are interactive and terminate quickly after execution.
 - Some are long running and run in the background (called services or daemons).
- View current running processes on the system using the **ps** comand (use -ax arguments to show all processes).
 - \$ **ps -ax**
- A handy way to look at the current resource utilization on the system is by using the **top** command.
 - \$ **top**
- Forcibly stop a process using the kill command.
 - \$ **kill** <process id>

Services

- Linux uses a service management program called **service** to manage the state of services.
 - **\$ sudo service start httpd**
 - Starts the Apache webserver on the system.
 - **\$ sudo service stop httpd**
 - Stops the Apache webserver on the system.
- A service can be configured to automatically start when a system starts up using the **chkconfig** command.
 - **\$ sudo chkconfig httpd on**

Package Management

- Every Linux system comes with applications pre-installed on the system in packages.
- Trying to figure out how to install and remove applications can be tricky because of package dependencies.
- A package management system is used to:
 - Track installed packages
 - Install new packages and dependencies
 - Remove packages
- A couple different package management systems exist and each distribution may use a different one.

Installing Packages

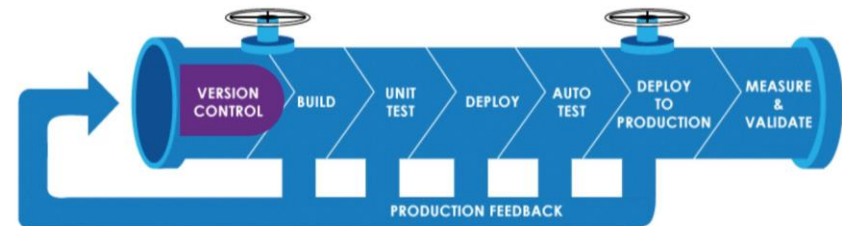
- Amazon Linux uses the common **yum** packaging system.
- Yum provides an easy way to update the currently installed packages on a system.
 - **\$ sudo yum update -y**
 - Kind of like running a Windows Update
- Install new applications on the system using the command:
 - **\$ sudo yum install httpd**
- Remove a package using the command:
 - **\$ sudo yum remove httpd**

Shutdown and Restart

- The shutdown command is used to shutdown a system (requires super-user privileges).
 - **\$ sudo shutdown**
- To tell the system to automatically restart after shutting down:
 - **\$ sudo shutdown -r now**
- Note, you can also use AWS tools and API to start/stop/restart Linux instances.

Automation

- Automation is a critical component of DevOps and modern IT practices.
 - Example: Software build pipeline
 - Changes are made to files in version control system.
 - New files are picked up by software delivery platform, built and tested.
 - Build artifacts are deployed to production systems.

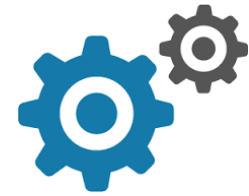


Automation goals

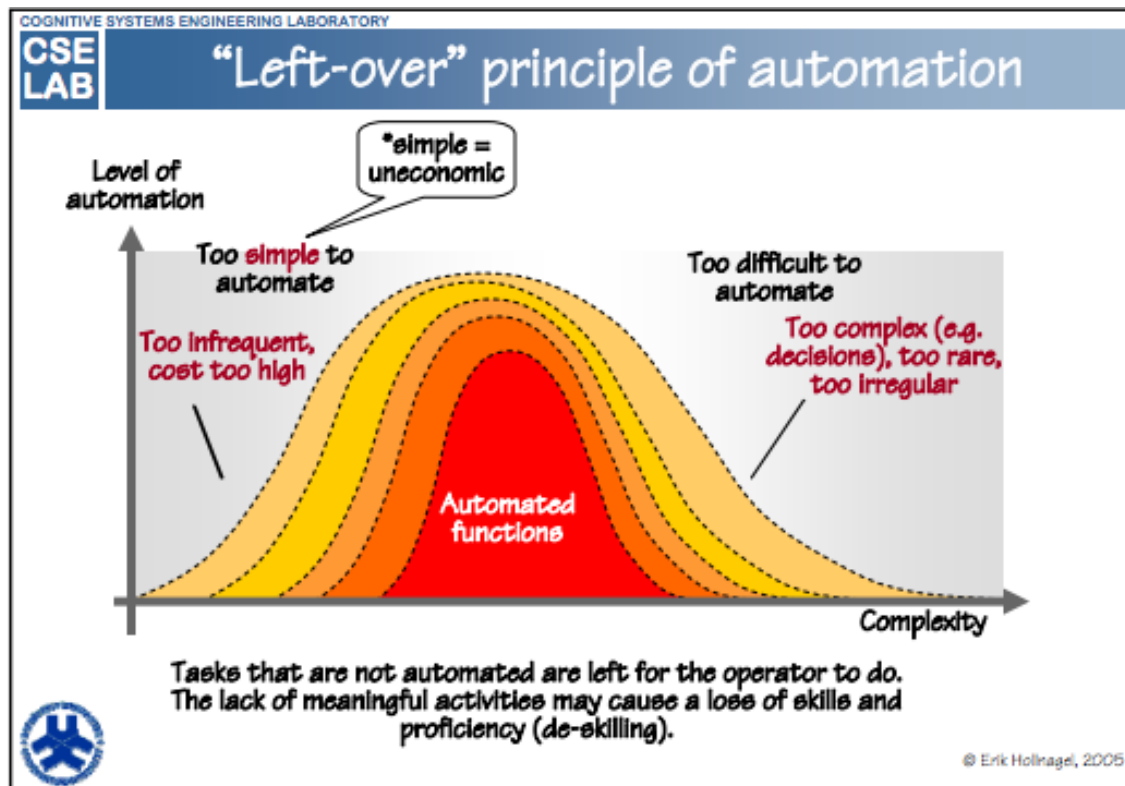


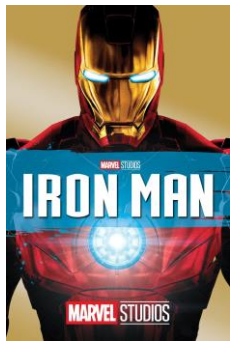
- **Help scaling:** one person performs the work of many.
- **Improve accuracy/ repeatability:** perform task same way every time.
- **Save time:** perform task faster than humans.
- **Make processes safer:** eliminate mistakes.
- **Empower users:** let's less experienced people perform complex tasks.

Automation

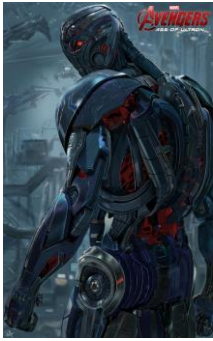


- Automation approaches:
 - Left-Over Principle: automate everything possible





Automation



– Compensatory Principle

- People aren't infinitely versatile machines.
- Machines do some things better than people and vice-versa.
- Example: Machines are better at polling remote hosts every 5 minutes for performance metrics.

– Complementarity Principle

- The more a system is automated, the less people understand how the system works (de-skilling).
- Try to strike a balance between automation and future growth in learning.

- Reading: <https://queue.acm.org/detail.cfm?id=2841313>

Automation

- Tool building vs. automation
 - Tools **improve a manual task** so that it can be done better.
 - Automation **eliminates the need** to perform a manual task.
 - Example: Manually running a code script vs. using a job scheduling server to trigger the running of a script.

Automation

- Building automation:
 - Majority of IT team's time should be spent on automation.
 - Identify and fix the biggest bottleneck first (Theory of Constraints).
 - Start small and work incrementally, don't try to automate everything at once.
- How to automate:
 - Shell scripts, scripting languages, configuration management, continuous integration system...

Shell Scripting

- Shell scripts are a common tool used for infrastructure automation.
 - Simple and quick to create.
 - Interpreted and executed by systems immediately.
 - Batch of commands grouped together in a file and executed from top to bottom.
 - Supported by wide variety of operating systems including Linux, Windows, and MacOS.

Shell Scripting

- We will focus on Linux shell scripts in this course using the BASH shell.
 - Released in 1989 and used as default shell for most Linux distributions.
- What is a shell?
 - An interactive command line processor that runs in a text window.
 - User can type in commands manually or the shell can read commands from a file (shell script).

Shell Scripting

- Example script:

```
#!/bin/bash
echo "Hello, world!"
echo "These are the current files, $PWD"
ls
exit 0
```

- Let's look at each line:
 - **#!/bin/bash**: pronounced "sha-bang-bin-bash", specifies which shell processor will execute the commands.
 - **echo**: prints text to the display terminal (aka standard output).
 - **ls**: command to list files.
 - **\$PWD**: a variable that contains the present working directory.
 - **exit**: terminate the script execution with a specified status code (0 = ok). Optional.

Shell variables

- Shell variables act like those found in typical programming languages.
 - Shells variables are not typed (i.e., no string, int, char, etc.)
- Define a variable using: `X="value"`
 - Note: Do not put spaces in assignment statement!
 - Common practice to capitalize bash variable names.
- Refer to the variable in a script using: `$X`
- Single quotes vs. double quotes
 - Variables are expanded (interpolated) within double quotes, but not single quotes.
 - `echo '$USER'` literally prints out the text: `$USER`
 - `echo "$USER"` prints out the value of the variable.

Shell variables

- The shell replaces every occurrence of a variable with its value in the script.

```
#!/bin/bash
LS="ls"
LS_FLAGS="-al"
$LS $LS_FLAGS
```

- The shell replaces the last line with the value of the variables (ls -al) and executes it!
- What if you need to display a variable followed by other characters?
echo "\$NAMExyz"
- Protect the variable by surrounding it with braces.
echo "\${NAME}xyz"

Shell variables

- Executing a shell script from the command-line:

***\$ scriptname* <arg1> <arg2> <arg3>**

Example: **\$ createreport orders 2017**

- Shell provides some built-in variables associated with this command execution:
 - **\$0**: scriptname (createreport)
 - **\$1**: arg1 (orders)
 - **\$2**: arg2 (2017)
 - **\$#**: number of command-line arguments (2)
- Other common built-in variables:
 - **\$HOME**: home directory of current user.
 - **\$HOSTNAME**: name assigned to the system.
 - **\$PATH**: file directories where executable applications are located.
 - **\$PWD**: current working directory.
 - **\$UID**: current user ID number.

Shell conditionals

- Shell supports conditional checks to branch execution:

```
if condition1
then
    statement1
    statement2
elif condition2
then
    statement3
else
    statement4
fi
```

- Statement #1 & #2 are executed if condition1 is true.
- The *condition* is typically written in the form: [operand1 operator operand2]

```
if [ $X -lt $Y ]          # if $X is less than $Y
if [ -n "$X" ]            # if $X is not empty
if [ -z "$X" ]            # if $X is empty
if [ $X = $Y ]            # if $X equals $Y
```

– Note: The spaces in the test bracket really matter!

Shell loops

- Loops allow the script to execute a series of commands multiple times
- for-loops:

```
for arg in <list>  
do  
    commands(s)...  
done
```

- Example:

```
for PLANET in "Mercury" "Venus" "Earth" "Mars"  
do  
    echo $PLANET  
done
```

Shell loops

- while loop

```
while [ condition ]  
do  
    command(s)...  
done
```

- Example:

```
#!/bin/bash  
X=0  
Y=5  
while [ "$X" -lt "$Y" ]  
do  
    echo "$X"  
    let X=X+1  
done  
exit 0
```

Shell loop globbing

- Anytime the shell sees a string containing an asterisk (*) it is replaced with a list of matching file names.

```
echo *.jpg      # list all jpeg files
```

- We can use this **globbing** action to our advantage:

```
#!/bin/bash
for X in *.jpg
do
    cp $X $X.bak      # Backup all the jpeg files
done
```

Shell command substitution

- Command substitution takes the output of a command and uses it as part of the statement.
 - Two methods: parenthesis or backtick

```
#!/bin/bash
```

```
FILES="$(ls)"
```

```
WEB_FILES=`ls /var/www/html`
```

```
echo $FILES
```

```
echo $WEB_FILES
```

Shell parameter substitution

- Parameter substitution is used to manipulate variable values
 - Uses braces {} to enclose variable construct

```
#!/bin/bash
VAR1=1
VAR2=2
echo ${VAR2-VAR1}          # output is 1
```

- Useful for assigning a default value to a variable using the :- characters

```
#!/bin/bash
echo "Name is ${NAME:-Sam}"  # name defaults to Sam
exit 0
```